

Phase 1: Docker Fundamentals

The TaskFlow Containerization Challenge

Mentee: _____

Mentor: _____

Assigned: _____ Target review date: _____

How this document is organized

This is Phase 1 of a two-phase track. You are containerizing a small, realistic multi-service application called TaskFlow using Docker and Docker Compose. Phase 2 (assigned separately, later) will take this exact application and deploy it to Kubernetes — so the decisions you make here about configuration, networking, and state will carry forward. Work through Parts 1–6 in order, produce the deliverables listed in Section 9, and bring your write-up and answers to reflection questions to your next mentoring session.

1. Learning Objectives

By the end of this task you should be able to demonstrate the following skills, unaided:

- Write a production-grade, multi-stage **Dockerfile** for a Node.js service
- Explain and apply image layering and build-cache principles to speed up builds
- Reduce image size and attack surface (small base images, non-root users, minimal layers)
- Orchestrate a multi-container application with **Docker Compose** (networks, volumes, environment variables, health checks, startup ordering)
- Externalize configuration and secrets instead of hard-coding them into images
- Persist stateful data safely across container restarts using volumes
- Diagnose and fix a broken multi-container stack using Docker's CLI tooling
- Understand image tagging & registries well enough to publish a versioned image

2. Prerequisites

- Docker Desktop (or Docker Engine) + Docker Compose v2 installed and working (docker version, docker compose version)
- Comfortable with a terminal / shell
- Basic familiarity with Node.js is helpful but not required — the application code is provided
- A GitHub or Docker Hub account (only needed for the bonus section)

3. The Scenario

TaskFlow is an early-stage startup building a simple team task-tracking tool. Right now, every developer runs the stack by hand: a locally installed PostgreSQL, a locally installed Redis, and the API started with `node server.js`. Onboarding a new developer takes half a day of installing the right versions of everything, and the team is constantly hitting “works on my machine” bugs.

You have joined as the engineer responsible for fixing this. Your task: containerize the entire stack so that *any* developer can bring up the full application with a single command, with no manual installs beyond Docker itself. The company has also told you, informally, that they intend to move this workload onto a Kubernetes cluster within the next few months — so you should avoid decisions that would make that migration harder than it needs to be.

4. Architecture

TaskFlow is made up of four independent, communicating services:

Service	What it is	Talks to
taskflow-web	Static HTML/JS front end, served by Nginx	taskflow-api (over HTTP)
taskflow-api	Node.js / Express REST API	taskflow-db, taskflow-cache
taskflow-db	PostgreSQL 16 — persistent store of tasks	—

Service	What it is	Talks to
taskflow-cache	Redis 7 — caches the task list for 30s	—

Request flow:

```

Browser  ---> | taskflow-web | (nginx, port 80)
               +-----+
               | fetch('/api/tasks')
               v
               +-----+         +-----+
               | taskflow-api | <----> | taskflow-cache | (redis, port 6379)
               | (node, :4000) |         +-----+
               +-----+
               | SQL
               v
               +-----+
               | taskflow-db | (postgres, port 5432)
               +-----+

```

Why this matters for what comes next

Every choice below is deliberately “boring” and standard, because that is exactly what maps cleanly onto Kubernetes primitives in Phase 2. Keep this table in mind as you work — you don’t need to build any of the right-hand column yet, but you should understand *why* your Docker setup is shaped the way it is:

What you build now (Docker)	What it becomes later (Kubernetes)
A container image per service	The image used inside a Pod’s container spec
A Compose service	A Deployment + a Service (for stable networking)
Service name used for inter-container DNS (e.g. taskflow-db)	Kubernetes Service DNS name (identical idea)
environment: values in compose	ConfigMap (non-secret) and Secret (secret) objects
A named volume for taskflow-db	A PersistentVolumeClaim bound to a PersistentVolume
HEALTHCHECK / healthcheck: in compose	livenessProbe and readinessProbe
depends_on with a health condition	Readiness gating between Pods (no direct equivalent — discussed later)
Host:container port mapping	A Service (ClusterIP) + optionally an Ingress
Image tag pushed to a registry	The image: field every Pod spec needs to pull from

5. Starter Application Code

Create the following project structure and files **exactly as given** — this is the application you are containerizing, not part of the assessment. Everything from Section 6 onward (Dockerfiles, .dockerignore, docker-compose.yml) is what *you* need to write.

```
taskflow/  
├── api/  
│   ├── package.json  
│   ├── server.js  
│   └── db/  
│       └── init.sql  
└── web/  
    ├── index.html  
    └── .env.example  
(you will add: api/Dockerfile, web/Dockerfile, .dockerignore, docker-compose.yml)
```

api/package.json

```
{  
  "name": "taskflow-api",  
  "version": "1.0.0",  
  "private": true,  
  "main": "server.js",  
  "scripts": { "start": "node server.js" },  
  "dependencies": {  
    "express": "^4.19.2",  
    "pg": "^8.11.5",  
    "redis": "^4.6.13",  
    "cors": "^2.8.5"  
  }  
}
```

api/server.js

```

const express = require('express');
const { Pool } = require('pg');
const { createClient } = require('redis');
const cors = require('cors');

const app = express();
app.use(cors());
app.use(express.json());

const PORT = process.env.PORT || 4000;

const pool = new Pool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT || 5432,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
});

const redisClient = createClient({
  url: `redis://${process.env.REDIS_HOST}:${process.env.REDIS_PORT || 6379}`
});
redisClient.on('error', (err) => console.error('Redis error:', err.message));

let redisReady = false;
redisClient.connect()
  .then(() => { redisReady = true; console.log('Connected to Redis'); })
  .catch((err) => console.error('Redis connect failed:', err.message));

app.get('/health', (req, res) => res.json({ status: 'ok' }));

app.get('/api/tasks', async (req, res) => {
  try {
    if (redisReady) {
      const cached = await redisClient.get('tasks');
      if (cached) return res.json({ source: 'cache', tasks: JSON.parse(cached) });
    }
    const result = await pool.query(
      'SELECT id, title, description, status, created_at FROM tasks ORDER BY id'
    );
    if (redisReady) {
      await redisClient.set('tasks', JSON.stringify(result.rows), { EX: 30 });
    }
    res.json({ source: 'db', tasks: result.rows });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to fetch tasks' });
  }
});

app.post('/api/tasks', async (req, res) => {
  const { title, description } = req.body;
  if (!title) return res.status(400).json({ error: 'title is required' });
  try {
    const result = await pool.query(
      'INSERT INTO tasks (title, description, status) VALUES ($1, $2, $3) RETURNING *',
      [title, description || '', 'pending']
    );
    if (redisReady) await redisClient.del('tasks');
    res.status(201).json(result.rows[0]);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to create task' });
  }
});

app.listen(PORT, () => console.log(`taskflow-api listening on port ${PORT}`));

```

api/db/init.sql

```
CREATE TABLE IF NOT EXISTS tasks (  
  id SERIAL PRIMARY KEY,  
  title VARCHAR(255) NOT NULL,  
  description TEXT,  
  status VARCHAR(50) DEFAULT 'pending',  
  created_at TIMESTAMP DEFAULT NOW()  
);  
  
INSERT INTO tasks (title, description, status) VALUES  
(  
  'Set up project repo', 'Initialize git and folder structure', 'done'),  
(  
  'Containerize the API', 'Write a production-grade Dockerfile', 'pending'),  
(  
  'Containerize the frontend', 'Serve static files via nginx', 'pending');
```

web/index.html

```
<!doctype html>  
<html>  
<head>  
  <meta charset="utf-8" />  
  <title>TaskFlow</title>  
</head>  
<body>  
  <h1>TaskFlow</h1>  
  <form id="new-task">  
    <input name="title" placeholder="Task title" required />  
    <input name="description" placeholder="Description" />  
    <button type="submit">Add</button>  
  </form>  
  <ul id="tasks"></ul>  
  
  <script>  
    const API_BASE = '/api';  
  
    async function loadTasks() {  
      const res = await fetch(`${API_BASE}/tasks`);  
      const data = await res.json();  
      const list = document.getElementById('tasks');  
      list.innerHTML = '';  
      data.tasks.forEach(t => {  
        const li = document.createElement('li');  
        li.textContent = `[${t.status}] ${t.title} (source: ${data.source})`;  
        list.appendChild(li);  
      });  
    }  
  
    document.getElementById('new-task').addEventListener('submit', async (e) => {  
      e.preventDefault();  
      const form = new FormData(e.target);  
      await fetch(`${API_BASE}/tasks`, {  
        method: 'POST',  
        headers: { 'Content-Type': 'application/json' },  
        body: JSON.stringify({  
          title: form.get('title'),  
          description: form.get('description')  
        })  
      });  
      e.target.reset();  
      loadTasks();  
    });  
  
    loadTasks();  
  </script>  
</body>  
</html>
```

.env.example

```
# copy this to .env and fill in real values -- never commit the real .env
DB_HOST=taskflow-db
DB_PORT=5432
DB_USER=taskflow
DB_PASSWORD=changeme
DB_NAME=taskflow
REDIS_HOST=taskflow-cache
REDIS_PORT=6379
PORT=4000
```

6. Your Tasks

Part 1 — Dockerize the API

Write `api/Dockerfile`. Requirements:

- Use a **multi-stage build**: an install/build stage, and a slim final stage that only contains what's needed to run the app
- Use a small base image (e.g. an alpine Node.js variant)
- Run the process as a **non-root user** inside the container
- Install only production dependencies in the final image (think about `npm ci` vs `npm install`, and `--omit=dev`)
- Order your `COPY` instructions to take advantage of Docker's layer cache (dependency manifests before source code)
- EXPOSE the port the app listens on
- Add a `HEALTHCHECK` instruction that calls `GET /health`

Checkpoint

Build the image, run `docker images` and note its size. Then try rewriting it with a non-alpine base and compare. You'll use these numbers in Part 4.

Part 2 — Dockerize the Frontend

Write `web/Dockerfile`. Requirements:

- Base it on an official nginx image
- Copy `web/index.html` into Nginx's default web root
- EXPOSE port 80
- **Bonus:** add a custom `nginx.conf` that reverse-proxies any request under `/api/` to the `taskflow-api` service, so the browser never needs to know the API's address

Part 3 — Orchestrate with Docker Compose

Write a `docker-compose.yml` that brings up all four services together. Requirements:

- All four services on a shared, explicitly-named network
- `taskflow-db`: use a **named volume** so task data survives `docker compose down && up`; mount `api/db/init.sql` into Postgres's `init` directory so the schema loads automatically on first boot
- `taskflow-api`: reads all connection details from environment variables (use an `.env` file — do not hard-code secrets into the compose file itself)
- Add `healthcheck`: blocks for **every** service (Postgres: `pg_isready`; Redis: `redis-cli ping`; API: `curl` its `/health` endpoint)
- Use `depends_on` with `condition: service_healthy` so the API doesn't start hammering the database before Postgres is actually ready
- Expose the frontend on host port 8080

Checkpoint

`docker compose up --build` should bring up the entire stack with one command. Open `http://localhost:8080`, add a task through the form, refresh, and confirm it persists. Then run `docker compose down` and up again — your task should still be there.

Part 4 — Image Hygiene & Optimization

- Write a proper `.dockerignore` for the API (exclude `node_modules`, `.git`, `.env`, logs, etc.)
- Record the final image size of `taskflow-api` with `docker images`, and inspect its layers with `docker history taskflow-api`
- In your write-up, produce a short before/after table: single-stage vs multi-stage, full node base vs `node:alpine`, with and without a proper `.dockerignore` — and 3–5 sentences explaining what changed and why

Part 5 — Debugging Challenge

A previous intern wrote the `docker-compose.yml` below and it does **not** work correctly. Some issues stop the stack from starting at all; others let it start but cause data loss, broken networking, or bad practices you shouldn't ship. There are at least **six distinct issues**. Get the stack fully working, and for each issue you fix, write one sentence explaining what was wrong and why your fix resolves it.

docker-compose.yml (BROKEN - fix this)

```
version: "3.9"
services:
  taskflow-db:
    image: postgres:latest
    environment:
      POSTGRES_USER: taskflow
      POSTGRES_PASSWORD: changeme
      POSTGRES_DB: taskflow
    ports:
      - "5432:5432"

  taskflow-cache:
    image: redis:7-alpine

  taskflow-api:
    build: ./api
    environment:
      DB_HOST: postgres
      DB_PORT: 5432
      DB_USER: taskflow
      DB_PASSWORD: changeme
      DB_NAME: taskflow
      REDIS_HOST: taskflow-cache
    ports:
      - "4000"
    depends_on:
      - taskflow-db

  taskflow-web:
    build: ./web
    ports:
      - "80:8080"
    depends_on:
      - taskflow-api
```

Useful commands while you debug: `docker compose logs -f <service>`, `docker exec -it <container> sh`, `docker inspect <container>`, `docker network ls` / `docker network inspect`.

Part 6 — Bonus (optional, stretch goals)

- Add `deploy.resources.limits/mem_limit` & `cpus` to each service — this maps directly onto Kubernetes `resources.requests/limits` later
- Tag your images with a real semantic version (e.g. `yourname/taskflow-api:1.0.0`) and push them to Docker Hub or GHCR — Kubernetes will need to pull from somewhere
- Split your compose file into a base `docker-compose.yml` plus a `docker-compose.override.yml` for local dev conveniences (e.g. live reload / bind mounts)
- Write a small bash smoke-test script that runs `docker compose up -d`, waits for health, curls `/api/tasks`, and reports pass/fail

7. Deliverables Checklist

- `api/Dockerfile` (multi-stage, non-root, healthcheck)
- `web/Dockerfile`
- `.dockerignore`
- A working, fixed `docker-compose.yml`
- `.env.example` filled in appropriately (real `.env` not committed)
- Short written report covering: image size before/after optimization, the bugs you found and fixed in Part 5 (one sentence each), and terminal output / screenshots showing the full stack running end-to-end (including a successful GET and POST to `/api/tasks`)
- Answers to the reflection questions (Section 9)
- (Bonus) pushed & tagged images, resource limits, override file, smoke-test script

8. Evaluation Rubric — for Mentor Use

Criterion	Weight	What to look for
Dockerfile quality	25%	Multi-stage, small final image, non-root user, sensible layer ordering, healthcheck present
Compose correctness	30%	Networking, named volumes, env management via <code>.env</code> , healthchecks + <code>service_healthy</code> conditions on every service
Debugging exercise	25%	Found & fixed at least 6 issues, correct explanations, stack genuinely works end-to-end afterward
Write-up & reflection	10%	Clear before/after numbers, thoughtful, specific answers (not generic definitions)
Bonus	+10%	Any bonus items attempted and working

Suggested format: walk through the debugging exercise together live and have him narrate each fix out loud — this surfaces gaps in understanding faster than reading the write-up alone.

9. Reflection Questions

Answer these in writing before your review session:

- ? Which of your configuration choices (env vars, service names, volumes) do you expect to carry over directly when this moves to Kubernetes, and which will need to change?
- ? Why doesn't Compose's `depends_on` + health condition guarantee your app is fully “ready” the way a Kubernetes readiness probe is meant to?
- ? If the `taskflow-db` container crashes and restarts, what happens to your data right now? What would you need to guarantee durability across multiple machines in a cluster?
- ? What trade-offs did you observe between image size, build time, and complexity when comparing single-stage vs multi-stage builds?

10. What's Next: Phase 2 Preview

Once this stack works end-to-end, Phase 2 will take the *exact same* container images and deploy them to a local Kubernetes cluster (kind or minikube). Your docker-compose.yml will be translated, piece by piece, into Deployments, Services, ConfigMaps, Secrets, PersistentVolumeClaims, and an Ingress — and you'll explore what Compose can't give you: self-healing, rolling updates, and horizontal scaling. Hang on to your images and this document.